

תרגיל 1 - חלק רטוב

המתרגל האחראי על התרגיל: מייס חורי.

שאלות על התרגיל – ב- Piazza בלבד.

הוראות הגשה:

- ההגשה בזוגות בלבד.
- עליכם לקרוא את נהלי תרגילי הבית תחת נהלי הקורס (הקובץ בעמוד "סילבוס" באתר הקורס)
- על כל יום איחור או חלק ממנו, שאינו באישור מראש, יורדו נקודות.
 - ניתן לאחר ב-3 ימים לכל היותר, ועל כל יום איחור יורדו 5 נקודות.
 - לבקשות מיוחדות להגשה באיחור יש לשלוח מייל למתרגל האחראי על התרגיל.
- הגשות באיחור רגיל (עם קנס) יתבצעו דרך אתר הקורס.
- יש להגיש את התרגיל בקובץ zip לפי ההוראות בעמוד הבא. **אי עמידה בהוראות אלו תעלה לכם בנקודות יקרות.**

הוראות והנחות לפתרון והגשת התרגיל

הוראות כלליות

- ישנם ארבעה סעיפים - הפתרון של כל אחד מהסעיפים צריך להופיע בקובץ נפרד ex1.asm, ex2.asm ובו' המצורפים לכם להשלמה.
- ההגשה שלכם צריכה להיות קובץ zip אחד שמכיל את ארבעת הקבצים שקיבלתם, מבלי שהורדתם מהם כלום וכאשר הוספתם להם אך ורק פקודות אסמבלי ולא שום דבר אחר.

טסטים

- מסופק לכם קובץ בדיקה לתרגיל 4 והוראות כיצד להשתמש בו. הטסט מכסה מקרה בסיסי ומסופק לכם על מנת שתדעו כיצד אנו מצפים לקבל את הפתרונות (בכל אחד מחמשת התרגילים) וכיצד הם יבדקו.
- מומלץ לבצע טסטים נוספים בעצמכם לכל חמשת התרגילים, על בסיס מבנה טסט זה.

חוקים לכתיבת הקוד

- בקוד שאתם מגישים **אין לשנות שום ערכים בזיכרון מלבד אלו שנאמר לכם מפורשות.**
- עליכם לבצע כל חישוב אפשרי ברגיסטרים. ביצוע חישובים על ערכים בזיכרון אינו יעיל.
- בכל סעיף הניחו כי בתוויות הפלט מוקצה מספיק מקום ואל תניחו כלום על תוויות המקור שלא כתוב במפורש.
- על כל תווית שתשמשו בה לצורך המימוש להסתיים בארבעת התווים _HW1.
 - לדוגמא –

loop: #this is a bad label name

loop_HW1: #this is a good label name

זכרו: אי עמידה בכל אחת מההוראות הנ"ל עלול לגרור הורדת ניקוד לתרגיל וכמובן להשפיע על זכותכם לערעור במקרים מסוימים.

אופן בדיקת התרגיל וכתיבת טסטים בעצמכם

כל ארבעת התרגילים יבדקו בצורה דומה (וגם הבונוס). לכן, עקבו אחר ההוראות שיתוארו להלן, על מנת לבדוק את הקוד שלכם לפני ההגשה. הפורמט המתואר מחייב ואי עמידה בו יוביל להורדת נקודות (שימו לב - לא בהכרח נוכל להתחשב בכך בערעור).

בכל תרגיל, תקבלו קובץ asm המכיל text.section. בלבד. עליכם להשלים את הקוד שם. אין להוסיף sections נוספים לקובץ בעת ההגשה (ההגשה חייבת להכיל **section text** בלבד. בפרט **לא** להכיל את **section data**).
(ההגשה חייבת להכיל section text בלבד. בפרט לא להכיל את section data).

אז איך בכל זאת תוכלו לבדוק את התרגיל שלכם?

זה פשוט. כאמור, לנוחותכם, צירפנו לתרגיל 4 טסט בודד ב-zip של קבצי הסטודנטים, היעזרו בו. הבדיקה היא בעזרת הקובץ run_test.sh שגם קיבלתם.

שימו לב שכל הטסטים על קבצי הקוד שלכם ירוצו עם timeout (כפי שגם מופיע ב-run_test.sh) ולכן כתבו אותם ביעילות. קוד שלא ייכתב ביעילות ולא יסיים את ריצתו על טסט מסוים עד ה-timeout, ייחשב כקוד שלא עמד בדרישות הטסט. בפרט הקוד ייבדק באותה מגבלת זמן שמופיעה ב-run_test.sh.

הערה:

לפני הרצת קבצי sh על המכונה, תצטרכו להריץ את הפקודה (פעם אחת בלבד):

```
chmod +x <your .sh file>
```

כדי לתת לקובץ הרשאות ריצה במערכת הפעלה.

הריצו את הקובץ run_test.sh:

```
./run_test.sh <path to asm file> <path to test file>
```

לדוגמה, עבור תרגיל 4 והטסט לדוגמה שלו, הריצו מתוך התיקייה שמכילה את קבצי הקוד של הסעיפים ואת תיקיית הטסטים את שורת הפקודה הבאה:

```
./run_test.sh ex4.asm ex4_example_test
```

כתיבת טסטים בעצמכם

לכל תרגיל תוכלו לכתוב טסט, שהמבנה שלו דומה למבנה של ex4_example_test, עם שינוי תוויות ובדיקות בהתאם.

שאלה 1: בעיית ספירה (10 נק.)

נתונה מחרוזת **null-terminated** בקידוד ASCII בתווית **str**.

יש לספור כמה אותיות אנגליות קטנות ('a'-'z' lower case) יש בה ולהחזיר את הכמות כ-**int** בתווית **count**.

שאלה 2: Balanced Brackets validator (30 נק.)

נתונה מחרוזת **null-terminated** בקידוד ASCII בתווית **exp**. המחרוזת מכילה רק את תווי הסוגריים הבאים:

[,], {, }, (,). עליכם לבדוק האם הסוגריים במחרוזת **מאוזנים**. את התשובה יש להחזיר כ-**int** בתווית הנתונה

valid. אם המחרוזת לא מאוזנת יש להחזיר 0, ואם כן אז יש להחזיר 1.

מה זה מחרוזת מאוזנת?

"An expression is balanced if each opening bracket has a corresponding closing bracket of the same type, the pairs are properly ordered and no bracket closes before its matching opening bracket"

הנחות לשם פשטות:

- המחרוזת מכילה רק את התווים של [,], {, }, (,).
- המחרוזת היא **null-terminated**.
- עומק הקיבון המקסימלי הוא לכל היותר 5.
- אין רווחים.

שימו לב: אסור שימוש ב **cpu stack** כדי לספור את הזוגות.

דוגמאות:

- "(){}[]" valid = 1.
- "({[]})" valid = 1.
- "([])" valid = 0.
- "" valid = 1.
- "((((" valid = 0.
- "{}" valid = 0.
- "([)" valid = 0. [כי יש לנו סוגר (לפני]

כלומר עבור שתי הדוגמאות הראשונות:

- Pairs = 3.
- Pairs = 3.
- "(){}", valid = 1 and pairs = 2.

שאלה 3: Encode Repeated Runs (20 נק.).

נתונה מחרוזת **null-terminated** בקידוד ASCII בתווית **str**. צריך לעבור על המחרוזת ולקודד אותה לפי הכללים שיוצגו בהמשך. את הקידוד יש לכתוב במחרוזת **null-terminated** אל תוך התווית הנתונה **enc** (מובטח כי יש מספיק מקום ב **enc**).

הנחות מקלות:

- אורך כל רצף הוא לכל היותר 9
- במחרוזת הנתונה בתווית **str** אין רווחים או מספרים, אך ורק אותיות אנגליות (גדולות וקטנות).

איך לקודד: כל רצף יקודד על ידי מספר החזרות ואחריו התו. **רצפים באורך 1 לא יכתבו כלל.**

דוגמאות:

- aaabbbccddddd תקודד ל 3a2b4d.
- abc תקודד למחרוזת ריקה.

שאלת בונוס *3 (5 נק.): Decode Repeated letters

נתונה מחרוזת **null-terminated** בקידוד ASCII בתווית **enc**. המחרוזת הנתונה מקודדת בזוגות מהצורה:

<Count><letter>

כאשר:

Count: היא ספרה רק מ 1 עד 9.

Letter: היא תו באנגלית קטנה או גדולה 'a'-'z' or 'A'-'Z'.

עליכם לפענח את המחרוזת ולכתוב את התוצאה כ-**null-terminated** לתווית **str**.

ניתן להניח שיש מספיק מקום עבור הפלט בתווית **str**.

דוגמאות:

Enc = "4a8c2D" -> str = "aaaaccccccccDD".

Enc = "1a3B2c" -> str = "aBBBcc".

Enc = "" -> str = "".

שאלה 4 (40 נק.): Lazer maze

נתונות התוויות עם המידע הבא:

- תווית **board** – מערך של char בגודל 8*8 המייצג לוח בגודל 8*8.
- תווית **lazer** – מערך בגודל 3 של int-ים, המכיל שלושה ערכים עבור row, col, direction.

כאשר direction מקודד:

direction	0	1	2	3
meaning	למעלה up	ימינה right	למטה down	שמאלה left

תווים היחידים האפשריים בלוח הם:

symbol	‘	’	‘	’	‘
meaning	תא ריק	קיר	מטרה	מראה סוג א'	מראה סוג ב'

הלייזר מתחיל במקום:

`lazer[0] = row, lazer[1] = col`

כשכיוונו מופנה לכיוון:

`lazer[2] = direction`

עליכם למלא את התווית **valid**, שגודלה **byte תא**, כך שאם הלייזר מגיע למטרה אז `valid = 1`, אחרת `valid = 0`.

מסלול הלייזר מחושב באופן הבא:

1. אם הלייזר יצא מגבולות הלוח צריך לכתוב 0 לתווית `valid`.
2. אם הלייזר פגע בקיר '#' אז צריך לכתוב 0 לתווית `valid`.
3. אם הלייזר הגיע למטרה 'D' צריך לכתוב 1 לתווית `valid`.
4. אם הלייזר הגיע לתא שמכיל ' ' (כלומר תא ריק) אז הוא ימשיך באותו הכיוון.
5. אם הלייזר הגיע למראה, הוא משנה את כיוונו באופן הבא:
מראה ' / :

- אם כיוונו היה **למעלה**, הוא ישנה את כיוונו **ימינה**
- אם כיוונו היה **ימינה**, הוא ישנה את כיוונו **למעלה**
- אם כיוונו היה **למטה**, הוא ישנה את כיוונו **שמאלה**
- אם כיוונו היה **שמאלה**, הוא ישנה את כיוונו **למטה**

מראה ' \ :

- אם כיוונו היה **למעלה**, הוא ישנה את כיוונו **שמאלה**
- אם כיוונו היה **שמאלה**, הוא ישנה את כיוונו **למעלה**
- אם כיוונו היה **ימינה**, הוא ישנה את כיוונו **למטה**
- אם כיוונו היה **למטה**, הוא ישנה את כיוונו **ימינה**

הנחות:

- הלוח הוא 8×8 .
- המערך בתווית board מסודר שורה לאחר שורה, כלומר, שמונת הבתים הראשונים לאחר תחילת board מייצגים את השורה הראשונה בלוח, שמונת הבתים הבאים מייצגים את השורה השנייה בלוח, וכן הלאה.
- Lazer[0] and lazer[1] תמיד בגבולות הלוח.
- Lazer[2] הוא תמיד מספר בין 0-3.
- הלוח יכול רק את התווים שתוארו בשאלה.
- אין צורך לזהות לולאות אין סופיות.
- המסלול של הלייזר תמיד מסתיים או בפגיעה בקיר, או ביציאה מהגבולות או מגיע למטרה.

בנוס (5 נק): זיהוי לולאות אין סופיות, אם הלייזר מגיע שוב לאותו מצב בדיוק. כלומר אותה שורה, אותה עמודה, ואותו כיוון.

דוגמאות:

1. בלוח הנתון למטה והמצב ההתחלתי $\text{lazer}[0]=3, \text{lazer}[1]=0, \text{lazer}[2]=1$ (כלומר $\text{lazer}=\{3,0,1\}$).
הלייזר מתחיל בשורה 3 ועמודה 0 וכיוונו הוא ימינה. המסלול שהלייזר יבצע: $(3,0) \rightarrow (3,1) \rightarrow (3,2)$
ואז משום שבתא $(3,2)$ ישנו הסימן T, הלייזר יגיע למטה ולכן יש להחזיר $\text{valid} = 1$.

.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
.	.	T	.	.	.	.	.
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.

2. עבור אותם תנאי התחלה כמו בדוגמה 1, והלוח:

.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
.	.	#	.	T	.	.	.
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.

הלייזר יבצע את אותו המסלול כמו בדוגמה 1, ויגיע לתא $(3,2)$, שם יש קיר '#', ולכן יוחזר $\text{valid} = 0$.

3. עבור אותם תנאי התחלה כמו בדוגמה 1, עבור הלוח:

.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.
.	.	T	.	.	.	.	.	.
.	.	/	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.

הלייזר יבצע תחילה את אותו מסלול כמו בדוגמה 1, ויגיע לתא (3,2), שם יש מראה '/' (מראה מסוג ב') ולכן כיוון הלייזר ישתנה מימין ללמעלה ולכן הלייזר יגיע ל T שנמצא בתא (2,2) ולכן יש להחזיר $valid = 1$.

כלומר, המסלול:

start at (3,0) then move right to (3,1) then again move right to (3,2) change direction because we have a mirror – right -> up and move up to (2,2).

(3,0) -> (3,1) -> (3,2) -> (2,2)

הערות אחרונות

איך בונים ומריצים לבד?

נתון לכם קובץ טסט לתרגיל 4 וקובץ `run_test.sh`. אתם יכולים לשנות את הקובץ הטסט של תרגיל 4 ולהשתמש במבנה שלו כדי לבדוק תרגילים אחרים באותה הצורה.

כדי להריץ, או לנפות שגיאות:

```
# run assembler (merge the 2 files into one asm before)
```

```
as ex4.asm ex4_example_test -o my_test.o
```

```
ld my_test.o -o ex4 #run linker
```

```
./ex4 #run the code
```

```
gdb ex4 #enter debugger with the code
```

את הקוד מומלץ לדבג באמצעות `gdb`. לא בטוחים עדיין איך? על השימוש ב-`gdb` תוכלו לקרוא עוד במדריך באתר הקורס. בהתחלה זה קשה, אבל זה יהיה שווה את זה! (ואם הסתבכתם – אנחנו זמינים בפיאצה).

- שימו לב – למכונה הווירטואלית של הקורס מצורפת תוכנת `SASM`, אשר תומכת בכתיבה ודיבוג של קוד אסמבלי וכן יכולה להוות כלי בדיקה בנוסף ל-`gdb`. (פגשתם אותה בתרגיל בית 0).
כתבו ב-`cmd`:

```
sasm <path_to_file>
```

כדי להשתמש ב-`SASM` לבנייה והרצת קבצי ה-`asm`, עליכם להחליף את שם התווית `start` בשם `main` (זאת מכיוון ש-`sasm` מזהה את תחילת הריצה על-ידי התווית `main`. **אל תשכחו להחזיר את `start` לפני ההגשה!**).

ניתן לדבג באמצעות מנגנון הדיבוג של `SASM` במקום עם `gdb`, אך השימוש בו על אחריותכם **כלומר לא נתמך על ידנו ולא נתחשב בבעיות בעקבותו בערעורים**. שימו לב לשוני בין אופן ההרצה ב-`SASM` לאופן ההרצה שאנו משתמשים בו בבדיקה שלנו).

- בטסט אתם תפגשו את השורות הבאות

```
movq $60, %rax
```

```
movq $X, %rdi # X is 0 or 1 in the real code
```

```
syscall
```

שורות אלו יבצעו `exit(X)` כאשר `X` הוא קוד החזרה מהתוכנית – 0 תקין ו-1 מצביע על שגיאה בקוד שאתם מגישים, אסור לפקודה `syscall` להופיע אצלכם בקוד. קוד שיכיל פקודה זו, יקבל 0.